

Classic CAN 환경을 위한 경량 고신뢰성 FOTA 시스템 설계

정보통신공학과 20213041 강태웅 지도교수: 강창순 교수님

FOTA(Firmware Over-The-Air)의 필수성과 저비용 Edge ECU의 역할

왜 FOTA가 필요한가?

SDV(Software Defined Vehicle)

"소프트웨어 중심 자동차(SDV)의 말단 연결성 확보"

결함 수정 및 리콜 방지

하드웨어 교체 없이 원격 버그 수정으로 막대한 리콜 비용 절감

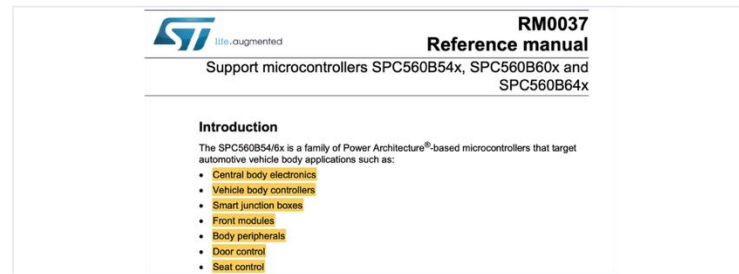
지속적 기능 개선

센서 알고리즘 업데이트를 통한 차량 성능 최적화 및 가치 증대

보안 취약점 보완

Classic CAN과 소프트웨어 보안 취약점을 소프트웨어 패치로 즉각 대응

저비용 Edge ECU의 활용처



"차량 전반에 퍼져있는 스마트 엣지 노드"

Body Control & Lighting

도어 모듈, 시트 조절기, 헤드램프 등 사용자 편의 및 조명 장치

Smart Sensors

TPMS, 주차 보조 센서 등 데이터 수집 및 말단 제어 노드
(Tire Pressure Monitoring System)

저비용 Edge ECU(Electronic Control Unit) 와 Classic CAN 환경



초소형 엣지 제어기

차량 내 곳곳에 존재하는 저비용 Edge ECU들의
공통적인 제한된 특성



제한적인 시스템 자원

RAM 및 Flash 메모리 자원이 매우 제한되어
복잡한 소프트웨어 스택을 적용하기 어려운 환경



구형 통신망 의존

고속의 최신 네트워크망이 아닌 대역폭이 좁고 (1Mbps)
데이터 페이로드 제약이 많은 Classic CAN 통신망 의존



Payload 8 Byte 한계

Classic CAN은 한 프레임당 최대 8 Byte payload만 지원
펌웨어 전송 시 수천 개 이상의 패킷 필요



제한된 대역폭과 우선순위 문제

1Mbps로 제한된 대역폭과 늘어나는 ECU와 데이터 트래픽으로
인한 CAN 버스의 과부하 문제



Firmware Risk

패킷 손실 → Firmware integrity 손상 → ECU Brick 위험

차량 네트워크 패킷 손실 메커니즘

① 전자기 간섭(EMI) 및 에러 전파

전자기 간섭으로 물리적 노이즈 발생:
인버터/점화 플러그 과도 전압에 의한
CAN 선로 비트 오류

Error Frame 연쇄 작용:
하드웨어 단의 프레임 파기 및 강제
재전송 유발
->통신 지연 누적

② 버스 고부하

우선순위 밀림 현상:
최상위 안전 제어 메시지에 밀려 펌웨어
패킷 송신 대기

프로토콜 타임아웃:
규정 대기 시간(N_Cr) 초과 시 소프트웨어
계층 통신 강제 종료 (Abort)

③ 수신 버퍼 오버플로우

Flash 기록 병목:
수신 ECU의 Flash Erase/Write 작업 시
일시적인 CPU 자원 점유

하드웨어 버퍼 Full:
ISR(인터럽트) 지연으로 인한 내부 CAN RX
FIFO 오버런
-> 물리적 패킷 Drop

Baseline: UDS/ISO-TP Firmware Update over CAN

상용 차량의 일반적으로 AUTOSAR 기반 통신 스택을 사용합니다.

Application Layer

UDS



Transport Layer

ISO-TP



Network Layer

CAN

 **UDS**

ISO 14229

진단 서비스 및 펌웨어 업데이트 명령(Download Request, Transfer Data 등)을 처리하는 상위 애플리케이션 프로토콜입니다.

 **ISO-TP**

ISO 15765-2

CAN의 8바이트 제한을 극복하기 위해 대용량 데이터를 분할(Segmentation)하고 흐름 제어(Flow Control)를 수행합니다.

 **CAN**

Controller Area Network

차량 내 제어기 간의 실제 물리적 통신을 담당하며, 높은 신뢰성과 실시간성을 보장하는 표준 네트워크 규격입니다.

ISO-TP (ISO 15765-2) 데이터 전송 메커니즘

SF (Single Frame)

7 Byte 이하의 데이터를 단일 프레임으로 전송할 때 사용됩니다.

FF (First Frame)

대용량 데이터 전송을 시작할 때, 전체 메시지 길이와 첫 번째 데이터 조각을 알리는 프레임입니다.

CF (Consecutive Frame)

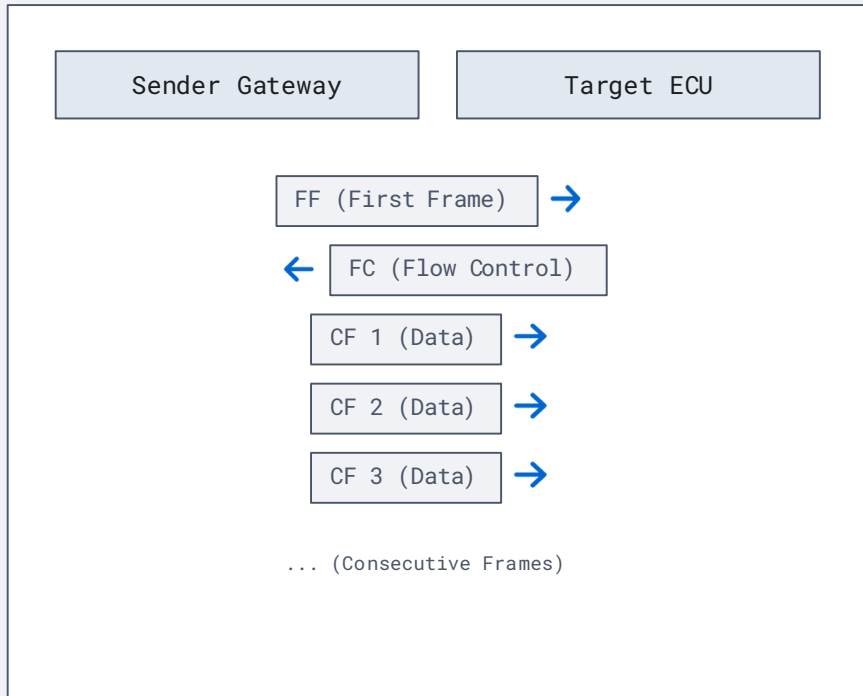
FF 이후, 분할된 데이터의 연속적인 조각들을 전송하는 프레임입니다. 각 프레임은 순서 번호를 포함해 전송합니다.

각 CF에는 4-bit Sequence Number (0-15)가 포함됩니다.

FC (Flow Control / BS, STmin)

수신 ECU가 송신자에게 전송 속도와 버퍼 상태를 알려, 데이터 오버플로우를 방지하는 제어 프레임입니다.

송신 속도를 제한하여 수신 ECU 버퍼 overflow를 방지



Packet Loss Problem in ISO-TP(Baseline)

⚠ 구조적 한계

- > ISO-TP는 엄격한 순차 전송을 전제로 설계되어, 프레임 순서가 어긋나면 즉시 오류로 간주합니다.
- > Sequence Number 불일치가 발생하면 ISO-TP 메시지 재조립이 실패하고 상위 계층(UDS)에서 해당 전송 세션이 중단됩니다.

[CF 1] → [CF 2] → [LOST] → [CF 4]
Result: ERROR_SEQUENCE_NUMBER

🔄 비효율적 복구 방식

- Sequence Number mismatch 발생 시 ISO-TP reassembly가
- > 실패하고 메시지가 폐기된다.
이후 상위 계층(UDS)이 TransferData block을 다시 요청하여 복구가 수행된다.
 - > 잦은 재전송은 네트워크 대역폭을 낭비하고, 전체 업데이트 시간을 늘립니다.

- ❌ ISO-TP Reassembly Failure
- 🔄 UDS TransferData Block Retransmission

Proposed Protocol: Selective NACK을 통한 효율적 오류 복구

✓ Packet 1: Success

✓ Packet 2: Success

✗ Packet 3: **Lost**

✓ Packet 4: Success

✓ Packet 5: Success

↩ Selective NACK Request: **Packet 3**

→ Gateway retransmits only Packet 3

↩ 선택적 재전송 (Selective NACK)

전체 펌웨어 블록 전송을 다시 시작하지 않고, 수신기가 누락된 특정 패킷만 식별하여 재전송을 요청하는 효율적인 오류 복구 메커니즘

🕒 통신 오버헤드 최소화

패킷 손실 시 세션 전체를 재시작해야 하는 ISO-TP 기반 업데이트 방식의 비효율을 해결

통신 메커니즘 비교: ISO-TP vs Proposed

ISO-TP (기준방식)

→ 256바이트 전송: FF → FC → CF

FF (First Frame) + 6바이트 데이터



FC (Flow Control) 수신자 응답



CF 01~15 (각 7바이트씩) | SN: 1-15

CF 16~30 (각 7바이트씩) | SN: 0-15

CF 31~36 (마지막 프레임들) | SN: 0-5

에러 시나리오: CF 23에서 에러 발생

CF 23 에러 → 256바이트 재전송

Selective NACK (제안방식)

256바이트 한 번에 전송 (Flow Control 없음)

CMD (Command Frame) + 6바이트 데이터

CF 01~15 (각 프레임 7바이트씩) | SN: 1-15

CF 16~30 (각 프레임 7바이트씩) | SN: 16-30

CF 31~36 (마지막 6개 프레임) | SN: 31-36

에러 시나리오: CF 23에서 에러 발생

CF 23 에러 → NACK 송신(비트맵 전송)

CF 23만 재전송 | CF 24-37은 이미 수신

기준방식은 1개의 패킷 손실이 블록 전체(256B) 재전송으로 이어지지만,
제안 방식은 손실된 패킷만 선택적으로 재전송하여 통신 오버헤드를 최소화한다.

Comparison: ISO-TP vs Proposed Protocol

Category	ISO-TP (Baseline)	Proposed Protocol
Packet Identification	4-bit Sequence Number (0-15)	6-bit Sequence Number (0-63)
Error Recovery	Block Retransmission (UDS TransferData)	Selective NACK
Retransmission Granularity	TransferData block 전체 재전송 필요	손실된 Packet만 재전송

Selective NACK 방식은 패킷 손실 시 **블록 단위 재시작**이 필요 없어
전송 시간과 통신 오버헤드를 단축할 수 있습니다.

Selective NACK 도입: 기회비용(Trade-off) 분석

+ 제안 방식이 얻는 것 (Efficiency)

- + 전송 효율 극대화: 손실 패킷 핀포인트 재전송
- + 네트워크 부하 경감: 불필요한 대역폭 감소
- + 업데이트 속도 향상: 패킷 손실 환경에서 전체 업데이트 시간 단축

- 제안 방식이 잃는 것 (Complexity & Cost)

- 표준 호환성 상실:
기존 진단 테스터 및 표준 스택과 AUTOSAR 툴체인과 직접 호환 불가
검증 및 디버깅을 위한 자체 모니터링 환경 구축 필요
- 검증 생태계(Toolchain) 부재:
Vector CANoe, Wireshark 등 기존 ISO-TP
표준을 지원하는 패킷 디코딩 툴 활용 제한

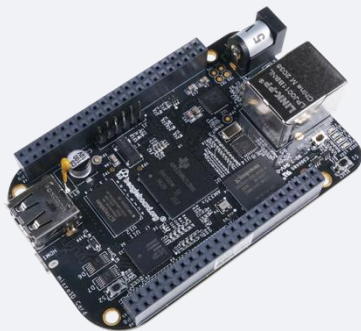


맞은 펌웨어 업데이트가 요구되는 SDV 특성상,
패킷 손실이 전체 업데이트 시간의 병목(Bottleneck)이 되지 않도록 개선

전체 하드웨어 구성



하웨이 LTE 모뎀



Beaglebone Black
Gateway 역할



CAN 버스 트랜시버 모듈
(SN65HVD230)



STM32F407 discovery
Target ECU

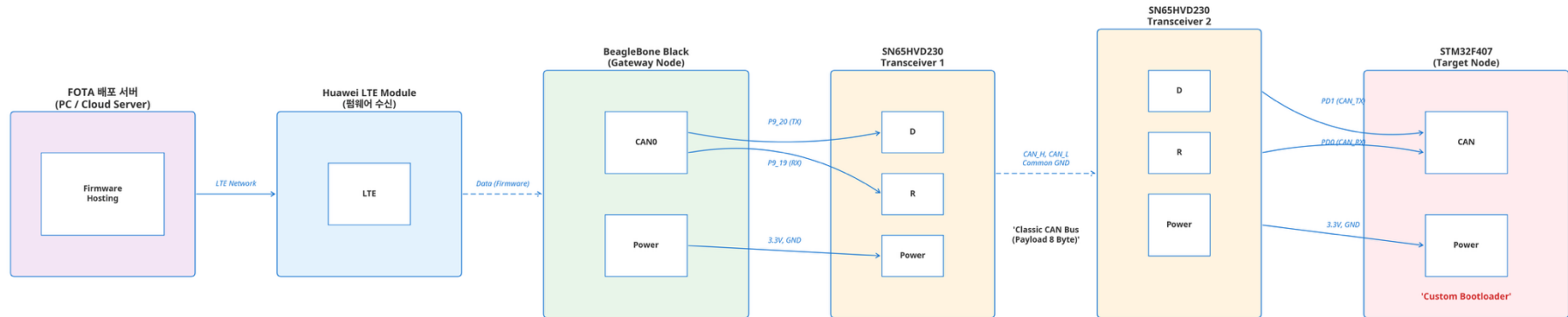


USB to UART
시리얼통신 디버깅용



STM32F103RB
Target ECU

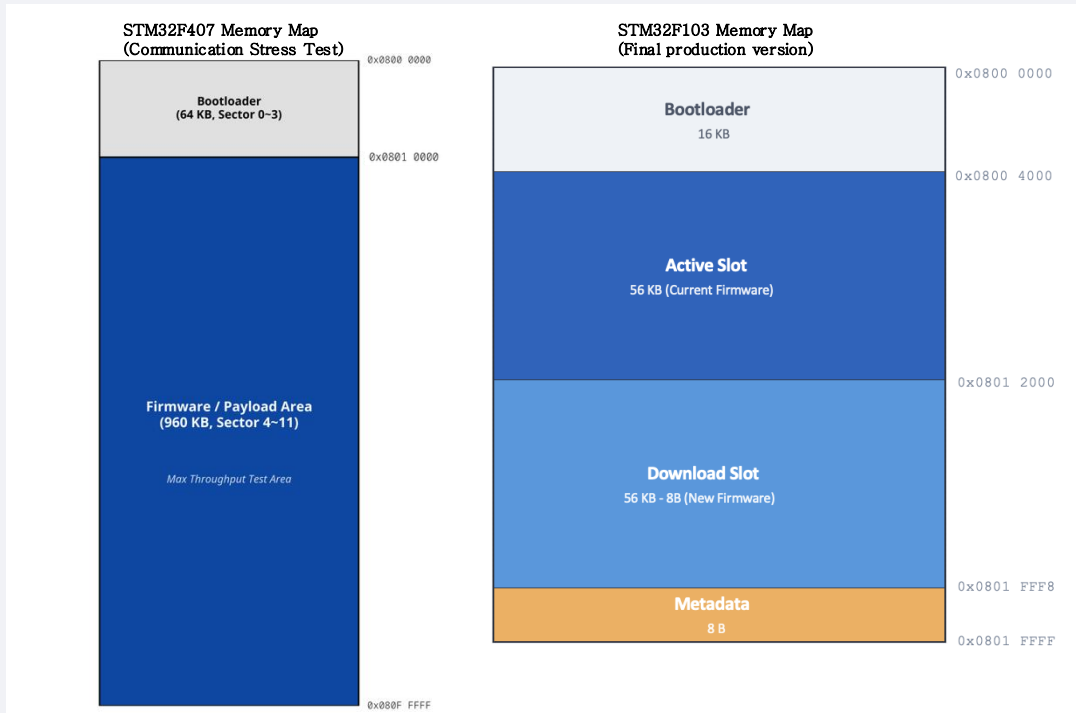
전체 하드웨어 시스템 구성도



전체 FOTA 시스템의 데이터 흐름 (Server PC -> LTE -> BeagleBone Black -> CAN 통신 -> STM32F407)

Target ECU인 STM32F407은 말단 노드로서 CAN 데이터를 수신하고 펌웨어를 업데이트함.

펌웨어 메모리 맵 및 부트로더 배치



1. 통신 스트레스 테스트용 (Single-slot)

목적: Selective NACK 프로토콜의 대용량 통신 성능
성능 및 복구율 검증

2. 최종 버전 (Dual slot)

목적: 안전한 펌웨어 업데이트 및 롤백(Rollback)
기능 구현

특징:

Active / Download Slot: A/B 파티션으로 분리하여
분리하여 업데이트 중 전원이 끊겨도 기존
펌웨어로 동작 보장 (안전성 확보)

Metadata Area: 펌웨어 버전 정보, 업데이트 상태,
상태, 무결성 검증 결과 등을 저장하여 부트로더가
부트로더가 부팅 시 참조

실험 환경 | 소프트웨어 결함 주입 (Sensitivity Analysis)

실험 구성

BBB (송신 측)

펌웨어 데이터 송신



Fault Injector

소프트웨어 패킷 드롭



STM32 (수신 측)

펌웨어 수신 · 플래싱

제안 프로토콜: FC 프레임 제거 + Selective NACK, N_Cr=150ms
ISO-TP: BS=256, STmin=0, N_Cr=150ms

테스트 조건

펌웨어 크기

64 · 128 · 256 · 512 KB

손실률 (Loss Rate)

0% · 0.01% · 0.05% · 0.1%

반복 횟수

각 조건 n=30 (Monte Carlo)

총 테스트 횟수

$4 \times 4 \times 30 = 960$ 회

CAN 속도

1 Mbps

전체 전송 시간 (Transfer Time)

손실 0% 베이스라인

FC 오버헤드로 ISO-TP가

64KB +26.3%, 512KB +7.2% 느림
(펌웨어 클수록 상대 격차 감소)

손실을 증가 시 격차 증가

0.1% 조건에서 ISO-TP:

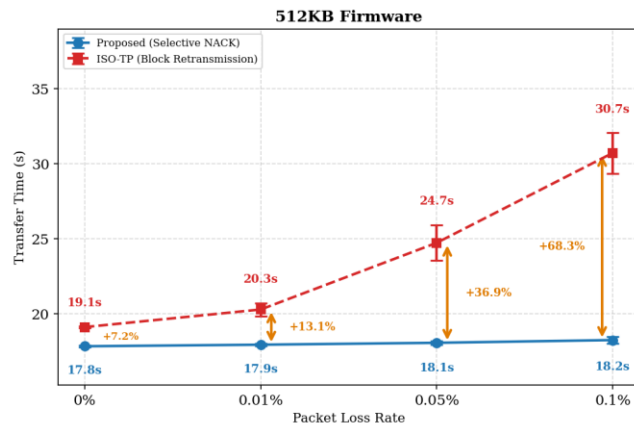
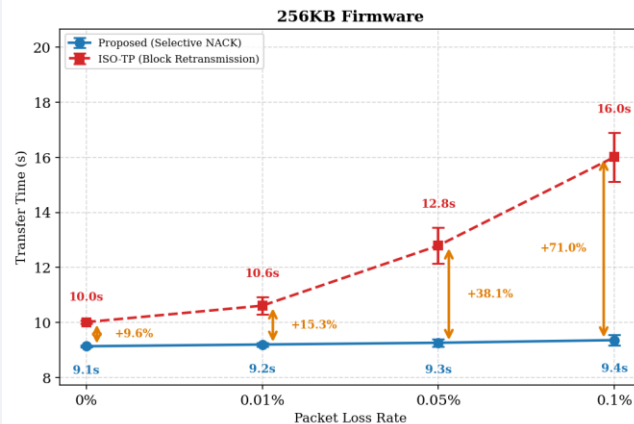
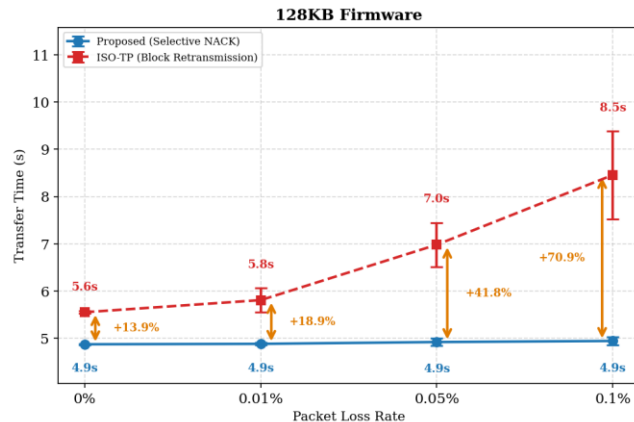
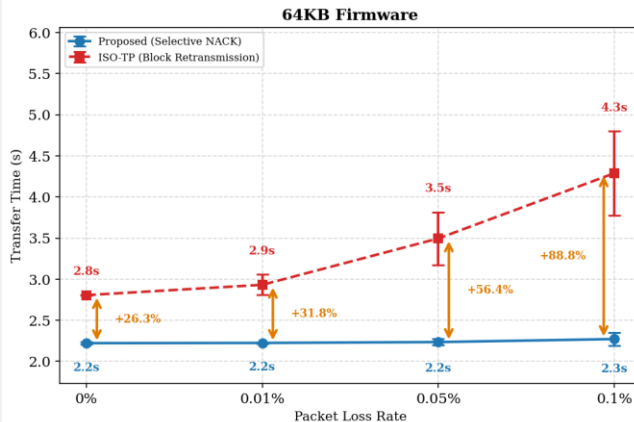
64KB +88.8%, 512KB +68.3%

Proposed은 거의 직선 유지

Selective NACK 효과

손실 프레임만 재전송하여
전송 시간 증가를 최소화

CAN Firmware Protocol Comparison: Transfer Time vs Packet Loss Rate
(n=30 trials, 1Mbps CAN, Error bars = $\pm 1\sigma$)



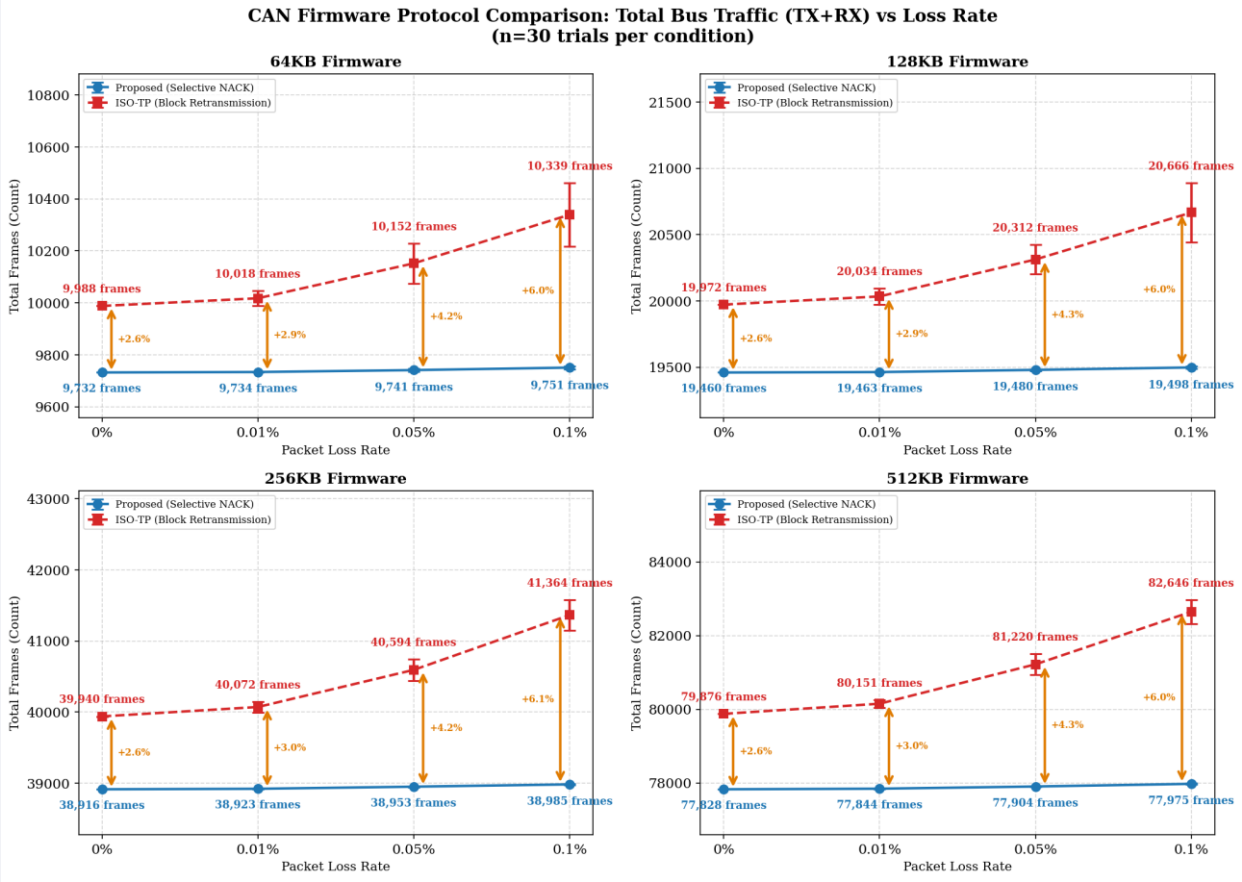
총 전송 프레임 (Total Bus Traffic)

손실률 증가 시 변화

ISO-TP는 블록 전체 재전송으로 총 프레임이 증가,
Proposed 은 손실 패킷 1개만 재전송하므로 총
프레임 수 증가 완만

반복적 FC(Frame Control) 프레임 제거

256B block마다 수행되는 FC handshake 제거로
누적 전송 오버헤드 감소



재전송 오버헤드 (Retransmission Overhead)

ISO-TP 블록 재전송 구조

BS=256 설정 → CF 1개 손실 시 256개(1,792B) 재전송

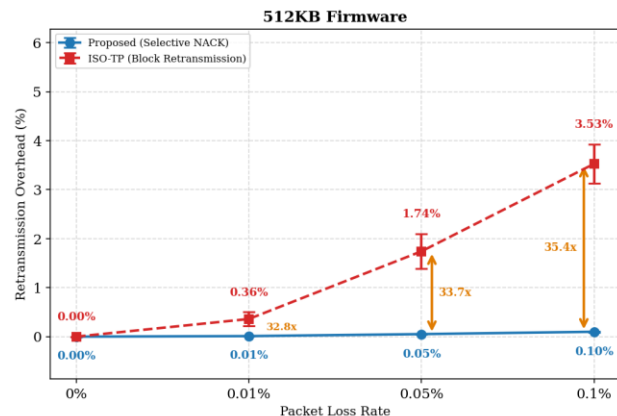
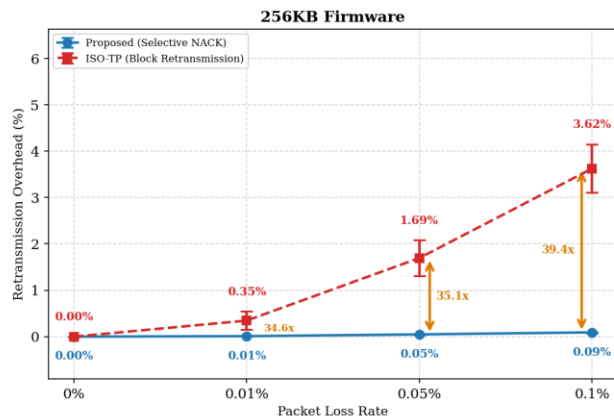
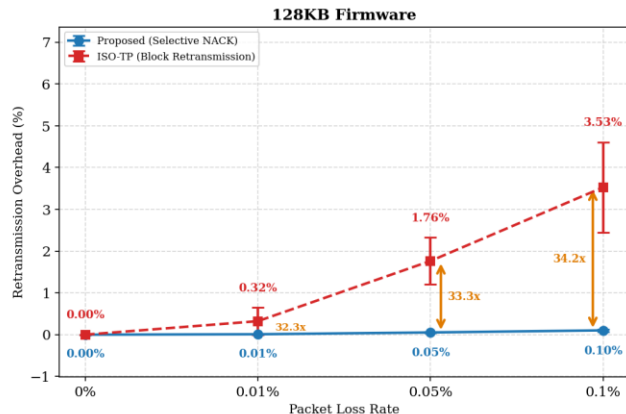
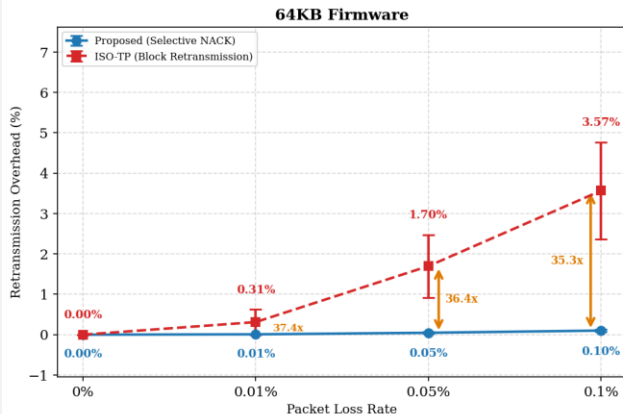
→ 0.1% 조건 256KB 기준 오버헤드 3.62%

Selective NACK 효과

손실 프레임(7B)만 재전송

→ 0.1% 조건 256KB 기준 오버헤드 0.09%

CAN Firmware Protocol Comparison: Retransmission Overhead vs Packet Loss Rate
(n=30 trials per condition)



실험 환경 설계 | 보드 변경 배경 및 스펙 비교

보드 변경 이유

상황

STM32F407 (고성능) 환경에서는 ISO-TP와 커스텀 프로토콜 간 성능 차이가 수치상 미미하게 나타남.

변경 이유

실제 차량의 ECU 환경(저사양 MCU)을 모사하기 위해 성능이 낮은 보드로 교체.

더 낮은 속도에서의 차이 확인.

→ 더 현실적이고 유의미한 비교 실험 가능

STM32F407VGT6

(기존 보드)

CPU	Cortex-M4, 168 MHz
Flash	1 MB
RAM	192 KB
Flash write	16 μ s
Program Size	32 bit
평가	과다 성능 → 차이 불명확

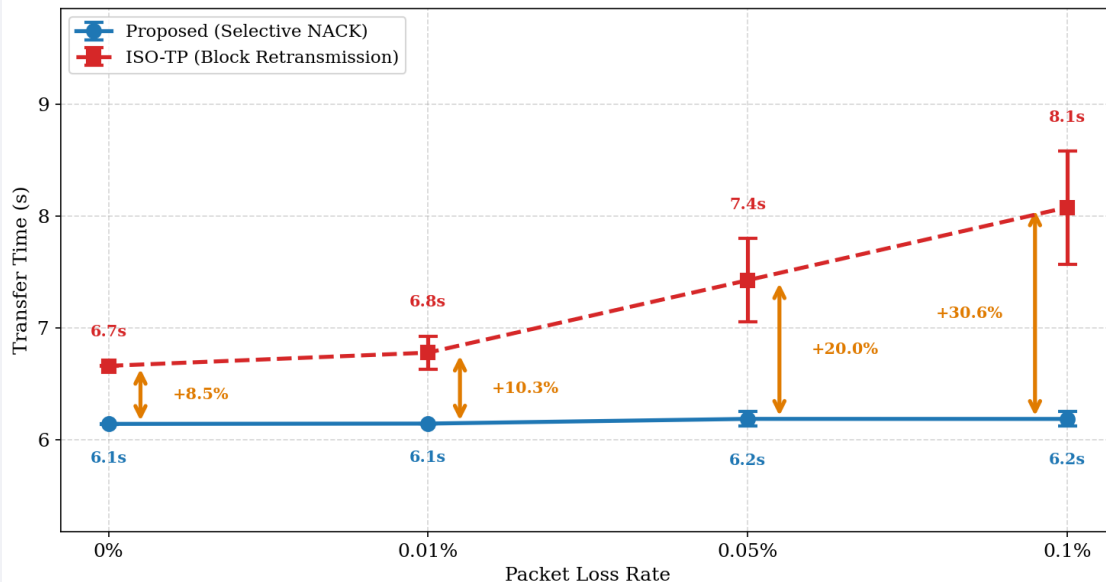
STM32F103RBT6

(신규 보드)

CPU	Cortex-M3, 72 MHz
Flash	128 KB
RAM	20 KB
Flash write	40 μ s ~ 52.5 μ s
Program Size	16 bit
평가	저사양 ECU 환경 모사

STM32F103RB 보드에서의 테스트

CAN Firmware Protocol Comparison: Transfer Time vs Packet Loss Rate
(64KB (STM32F103 Max) Firmware, 500kbps CAN, n=30 trials, $\pm 1\sigma$)



💡 결과

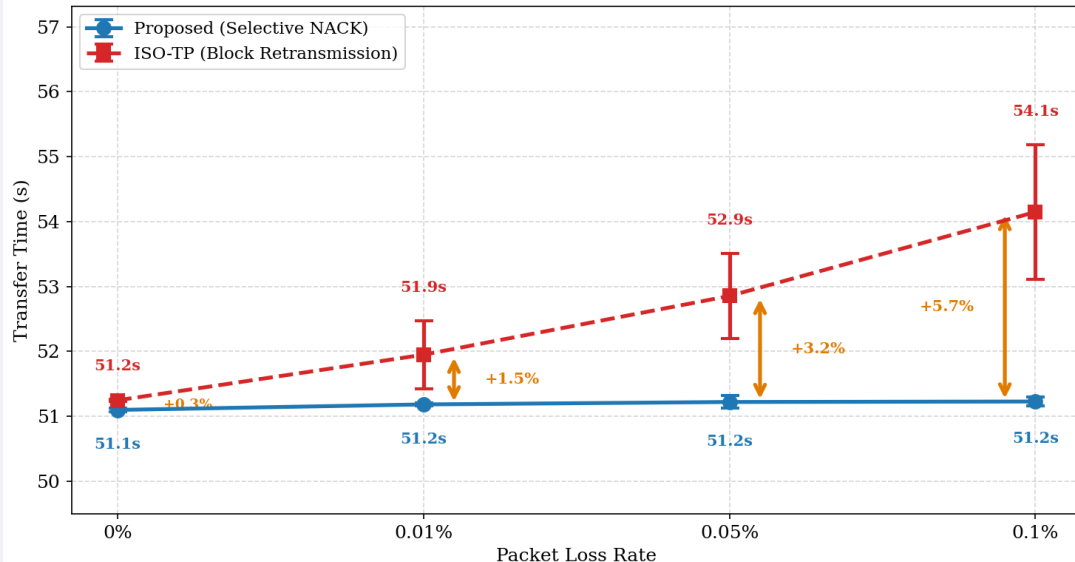
64KB에서 2.2s -> 6.1s 로 늘어남.

64KB를 전송하는데 기본적으로 많은 시간이 걸리지 않는다.

재전송으로 발생하는 시간 초 차이가 크지 않음.

STmin(Minimum Separation Time) = 5ms 테스트

CAN Firmware Protocol Comparison: Transfer Time vs Packet Loss Rate
(64KB (STmin=5ms) Firmware, 500kbps CAN, n=10 trials, $\pm 1\sigma$)



스트레스 테스트

구형 ECU의 처리 지연(STmin)과 네트워크 불안정성이 결합된 최악의 시나리오에서 프로토콜 성능 테스트

결과 분석

STmin = 5ms

STmin 증가로 전체 전송 시간이 크게 늘어남.

패킷 손실 상황

STmin의 증가로 전체 전송시간은 늘어났지만 제안 방식으로 개선되는 시간 차가 크지 않음.

결론

본 실험이 증명한 것

Selective NACK의 구조적 우위

동일 손실 조건에서 블록 재전송 대비 재전송 오버헤드 절감
→ 프로토콜 recovery 구조 자체의 차이

반복적 FC(Frame Control) 프레임 제거 효과

손실 0% 조건에서도 256B block마다 수행되는
FC handshake 제거로 누적 통신 오버헤드 감소

Classic CAN 환경 효율 개선 가능성

하드웨어 변경 없이 프로토콜 계층
최적화만으로 전송 효율 향상 확인

본 실험이 증명하지 못한 것

절대 전송 시간 개선이 제한적

64KB: 약 2.7초 단축 (4초 vs 6초)
→ 다중 ECU, 대용량 펌웨어, CAN-FD에서 테스트 필요

다중 제어기 병렬 업데이트 환경 검증

게이트웨이와 단일 제어기 간의 1:1 통신 환경을 기준으로 성능을 측정.
혼합 트래픽 환경에서의 arbitration 지연과 bus load 변화에 대한
검증 필요

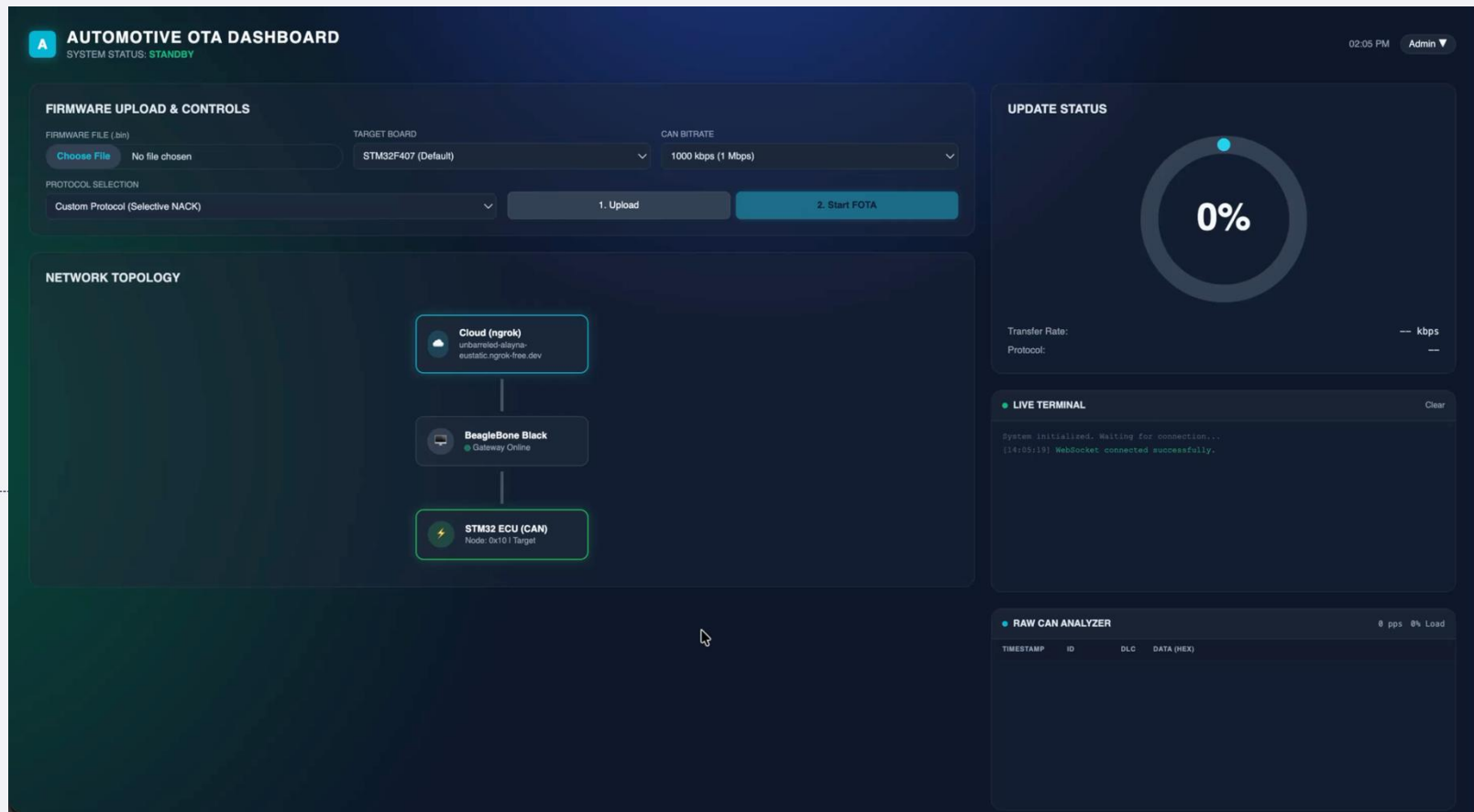
고정 블록 크기(BS) 및 NACK 비트맵 구조의 확장성 한계

현재 제안한 프로토콜의 Selective NACK 비트맵과 6비트 시퀀스
넘버(Max 64)는 256B 블록(37프레임)에 맞춰 복구가 가능하도록
최적화됨.

실차 BER 직접 재현 아님

손실률 0~0.1%는 실차 평균 BER이
아닌 Fault Injection 기반 Sensitivity Analysis 설계 환경

통합 시스템 아키텍처 및 실시간 관제 인터페이스 구현 결과



개발 환경



Host OS & Virtualization

UTM (Linux Ubuntu)
Debian(비글본 블랙 커널)



MCU Development/ Web

STM32CubeMX
STM32CubeIDE for VSCode
Backend: FastAPI / ngrok



Build System & Editor

Cmake
Visual Studio Code